

HomeWork3

Aswath Oruganti

June 28, 2026

1 Concept Questions

1. EM Algorithm for GMM: Derivation of τ_{ik}

In the Expectation-Maximization (EM) algorithm for a Gaussian Mixture Model (GMM), the E-step requires us to calculate the “responsibility” τ_{ik} . This is the posterior probability that the i -th data point x_i was generated by the k -th Gaussian component.

1.1 Define the GMM Component

A GMM with K components is defined by three sets of parameters for each component k :

- **Prior probability (mixing weight) π_k :** The overall probability that any given point belongs to cluster k , subject to $\sum_{k=1}^K \pi_k = 1$.
- **Mean vector μ_k and Covariance matrix Σ_k :** These define the k -th Gaussian distribution.

The likelihood of observing a data point x_i given it came from component k is the multivariate Gaussian probability density function (PDF):

$$P(x_i | z_i = k) = \mathcal{N}(x_i | \mu_k, \Sigma_k)$$

where $z_i \in \{1, \dots, K\}$ is the latent variable indicating the true cluster assignment.

1.2 Apply Bayes' Rule

Bayes' theorem is defined as $P(A | B) = \frac{P(B|A)P(A)}{P(B)}$. Mapping our GMM variables to Bayes' rule to find $\tau_{ik} = P(z_i = k | x_i)$:

$$\tau_{ik} = \frac{P(x_i | z_i = k)P(z_i = k)}{P(x_i)}$$

1.3 Expand the Terms

The Numerator (Joint Probability): The probability of choosing cluster k multiplied by the probability of drawing x_i from that specific cluster:

$$P(x_i | z_i = k)P(z_i = k) = \mathcal{N}(x_i | \mu_k, \Sigma_k)\pi_k$$

The Denominator (Marginal Likelihood): By the Law of Total Probability, we sum the joint probabilities over all K components:

$$P(x_i) = \sum_{j=1}^K P(x_i | z_i = j)P(z_i = j) = \sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)$$

1.4 Final Closed-Form Expression

Substituting the expanded numerator and denominator back yields the closed-form solution for the E-step:

$$\tau_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

2. Maximum Likelihood Estimators for a Multivariate Gaussian

Let $x_1, \dots, x_m$ be a set of m i.i.d. random vectors drawn from an n -dimensional Gaussian distribution $\mathcal{N}(\mu, \Sigma)$.

2.1 Formulating the Log-Likelihood Function

The probability density function (PDF) for a single observation x_i is:

$$p(x_i | \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x_i - \mu)^T \Sigma^{-1} (x_i - \mu) \right)$$

Because the observations are i.i.d., the joint likelihood function $\mathcal{L}(\mu, \Sigma)$ is the product of the individual probabilities:

$$\mathcal{L}(\mu, \Sigma) = \prod_{i=1}^m p(x_i | \mu, \Sigma)$$

To make the derivation easier, we take the natural logarithm of the likelihood function to get the log-likelihood function, $\ell(\mu, \Sigma)$:

$$\ell(\mu, \Sigma) = \ln \mathcal{L}(\mu, \Sigma) = \sum_{i=1}^m \ln p(x_i | \mu, \Sigma)$$

$$\ell(\mu, \Sigma) = \sum_{i=1}^m \left[-\frac{n}{2} \ln(2\pi) - \frac{1}{2} \ln |\Sigma| - \frac{1}{2} (x_i - \mu)^T \Sigma^{-1} (x_i - \mu) \right]$$

Pulling the terms that do not depend on the summation index i out of the sum gives:

$$\ell(\mu, \Sigma) = -\frac{mn}{2} \ln(2\pi) - \frac{m}{2} \ln |\Sigma| - \frac{1}{2} \sum_{i=1}^m (x_i - \mu)^T \Sigma^{-1} (x_i - \mu)$$

2.2 MLE for the Mean ($\hat{\mu}$)

To find the maximum likelihood estimate for the mean, we take the partial derivative of the log-likelihood with respect to the vector μ and set it to zero.

Using the vector calculus identity $\frac{\partial}{\partial s} (x - s)^T W (x - s) = -2W(x - s)$ for a symmetric matrix W :

$$\frac{\partial \ell}{\partial \mu} = -\frac{1}{2} \sum_{i=1}^m [-2\Sigma^{-1}(x_i - \mu)]$$

$$\frac{\partial \ell}{\partial \mu} = \Sigma^{-1} \sum_{i=1}^m (x_i - \mu)$$

Setting this derivative to zero to find the maximum:

$$\Sigma^{-1} \sum_{i=1}^m (x_i - \mu) = 0$$

Assuming Σ is positive definite (and therefore invertible), we can multiply both sides by Σ , which eliminates it:

$$\sum_{i=1}^m (x_i - \mu) = 0$$

$$\sum_{i=1}^m x_i - \sum_{i=1}^m \mu = 0$$

$$\sum_{i=1}^m x_i - m\mu = 0$$

Solving for μ yields the sample mean:

$$\hat{\mu} = \frac{1}{m} \sum_{i=1}^m x_i$$

2.3 MLE for the Covariance Matrix ($\hat{\Sigma}$)

To derive the MLE for the covariance matrix directly with respect to Σ , we first substitute the estimated mean $\hat{\mu}$ back into our log-likelihood function. Grouping the constant terms into C , we have:

$$\ell(\Sigma) = C - \frac{m}{2} \ln |\Sigma| - \frac{1}{2} \sum_{i=1}^m (x_i - \hat{\mu})^T \Sigma^{-1} (x_i - \hat{\mu})$$

Next, we take the partial derivative of the log-likelihood with respect to the covariance matrix Σ . To do this, we rely on two standard matrix calculus identities for a symmetric matrix Σ :

1. The derivative of the log determinant: $\frac{\partial}{\partial \Sigma} \ln |\Sigma| = \Sigma^{-1}$
2. The derivative of an inverse quadratic form: $\frac{\partial}{\partial \Sigma} (a^T \Sigma^{-1} a) = -\Sigma^{-1} a a^T \Sigma^{-1}$

Applying these identities (where the vector $a = x_i - \hat{\mu}$), we differentiate the log-likelihood:

$$\frac{\partial \ell}{\partial \Sigma} = -\frac{m}{2} \Sigma^{-1} - \frac{1}{2} \sum_{i=1}^m [-\Sigma^{-1} (x_i - \hat{\mu})(x_i - \hat{\mu})^T \Sigma^{-1}]$$

Set the derivative to the zero matrix to find the maximum:

$$-\frac{m}{2} \Sigma^{-1} + \frac{1}{2} \sum_{i=1}^m \Sigma^{-1} (x_i - \hat{\mu})(x_i - \hat{\mu})^T \Sigma^{-1} = 0$$

To isolate Σ , we can pre-multiply and post-multiply the entire equation by Σ . Since $\Sigma \Sigma^{-1} = I$ (the identity matrix), this effectively clears the inverses:

$$-\frac{m}{2} \Sigma \Sigma^{-1} \Sigma + \frac{1}{2} \Sigma \Sigma^{-1} \left(\sum_{i=1}^m (x_i - \hat{\mu})(x_i - \hat{\mu})^T \right) \Sigma^{-1} \Sigma = 0$$

Simplifying this yields:

$$-\frac{m}{2}\Sigma + \frac{1}{2}\sum_{i=1}^m (x_i - \hat{\mu})(x_i - \hat{\mu})^T = 0$$

Rearranging the terms to solve for Σ :

$$\begin{aligned}\frac{m}{2}\Sigma &= \frac{1}{2}\sum_{i=1}^m (x_i - \hat{\mu})(x_i - \hat{\mu})^T \\ m\Sigma &= \sum_{i=1}^m (x_i - \hat{\mu})(x_i - \hat{\mu})^T\end{aligned}$$

Dividing by m gives the final closed-form sample covariance:

$$\hat{\Sigma} = \frac{1}{m}\sum_{i=1}^m (x_i - \hat{\mu})(x_i - \hat{\mu})^T$$

2.4 Discussion on the Rank of $\hat{\Sigma}$

When $m < n$

Let X_c be an $n \times m$ matrix where each column is a mean-centered data point $(x_i - \hat{\mu})$. We can express the estimated covariance matrix as a matrix multiplication:

$$\hat{\Sigma} = \frac{1}{m}X_cX_c^T$$

We can view this matrix multiplication as a sequence of linear transformations. The operation $X_cX_c^T$ maps data from $\mathbb{R}^n$ down into $\mathbb{R}^m$ (via X_c^T), and then back up into $\mathbb{R}^n$ (via X_c). Because the intermediate space $\mathbb{R}^m$ acts as an “information bottleneck” where $m < n$, the final output can span at most m independent dimensions. Therefore, the rank of the product is bottlenecked by the inner dimension:

$$\text{Rank}(\hat{\Sigma}) = \text{Rank}(X_cX_c^T) = \text{Rank}(X_c) \leq \min(n, m)$$

Because $m < n$, the maximum possible rank of X_c is m . However, we must also consider that the columns of X_c are mean-centered. By definition, the sum of all columns in X_c equals the zero vector ($\sum(x_i - \hat{\mu}) = 0$). This linear dependency means we lose one degree of freedom.

Therefore, when $m < n$, the rank of $\hat{\Sigma}$ is at most $m - 1$. Because $m - 1 < n$, $\hat{\Sigma}$ is guaranteed to be rank-deficient (singular), meaning its determinant is zero and it cannot be inverted.

Extreme Case: $m = 1$

If we only have one sample, x_1 , the maximum likelihood estimate for the mean is simply the data point itself: $\hat{\mu} = x_1$. When we calculate the covariance:

$$\hat{\Sigma} = \frac{1}{1}(x_1 - x_1)(x_1 - x_1)^T = \mathbf{0}$$

Because the resulting matrix is a matrix of all zeros, **the rank of $\hat{\Sigma}$ when $m = 1$ is exactly 0.**

Conclusion: When dealing with high-dimensional data, if your sample size m does not strictly exceed your feature dimension n (specifically, you need $m > n$), your sample covariance matrix will be singular. This requires specialized techniques—such as Principal Component Analysis (PCA) for dimensionality reduction, or adding a regularization term to the diagonal elements (Shrinkage/Ridge)—to make the matrix invertible for downstream tasks like classification or calculating probabilities.

2 Implementing EM for MNIST Dataset

1. EM Algorithm Implementation Analysis

1.1 EM Algorithm Implementation and Initialization Analysis

1. Dimensionality Reduction (Preprocessing):

Before initializing the Expectation-Maximization (EM) algorithm, the original 784-dimensional dataset of digits “2” and “6” was projected down to $n = 4$ dimensions using Principal Component Analysis (PCA). This step is crucial because calculating the inverse and determinant of a 784×784 covariance matrix at every iteration of the EM algorithm is highly computationally expensive and prone to numerical instability (singular matrices).

2. Initialization Strategy:

The EM algorithm is highly sensitive to its initial parameters because it optimizes a non-convex objective function, making it susceptible to local optima. The specific initialization constraints provided in the prompt were implemented as follows:

- **Mean Initialization (μ_k):** The means for the two components were initialized as standard normal random vectors: $\mu_1, \mu_2 \sim \mathcal{N}(0, I_4)$. Because the data was pre-processed using PCA, it is already mean-centered at zero with unit-scale variance along its principal components. Therefore, initializing the cluster centers near the origin $(0, 0, 0, 0)$ ensures they start in the densest region of the data cloud, allowing the EM algorithm to smoothly pull them toward the respective mass centers of digits “2” and “6”.
- **Covariance Initialization (Σ_k):** The covariance matrices were initialized using the formula $\Sigma_k = S_k S_k^T + I_n$, where S_k is a 4×4 matrix populated with standard normal random variables.
 - *Mathematical Justification:* A valid covariance matrix must be **symmetric and positive definite**. Simply generating a random matrix does not guarantee this. By calculating $S_k S_k^T$, we construct a symmetric, positive semi-definite matrix.
 - *The Role of I_n :* Adding the identity matrix I_n acts as a strict regularization term (similar to Ridge regularization). It ensures that all eigenvalues of the initial covariance matrix are strictly greater than or equal to 1. This prevents a classic failure mode in GMMs known as **covariance collapse**, where a Gaussian component shrinks infinitely tightly around a single data point, causing its determinant to approach zero and the log-likelihood to explode to infinity.

3. Execution and Numerical Stability:

During the implementation of the E-step, directly calculating Gaussian probabilities $p(x_i | \mu, \Sigma)$ can result in severe numerical underflow (probabilities rounding down to exactly 0.0), causing division-by-zero errors when updating the weights. To prevent this, the algorithm evaluates the densities in the logarithmic domain and utilizes the **log-sum-exp trick**. This guarantees mathematical stability when calculating the responsibilities (τ_{ik}) across all iterations.

1.2 Log-Likelihood Convergence Analysis

The Expectation-Maximization algorithm guarantees that the log-likelihood of the observed data will monotonically increase (or remain flat) with every successive iteration. It will never decrease.

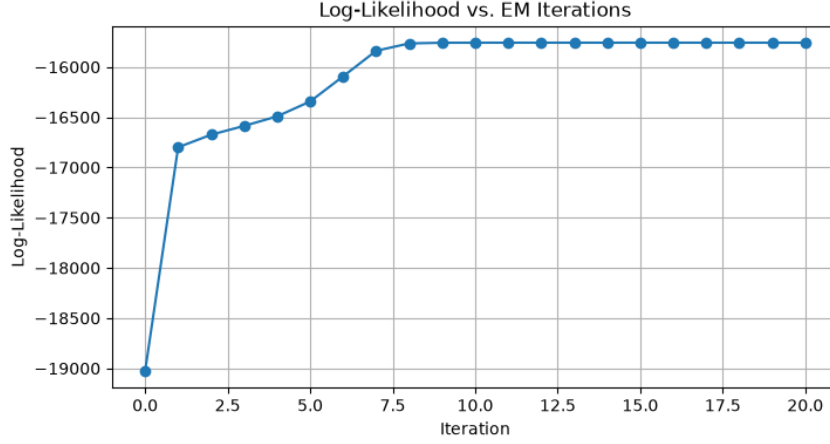


Figure 1: Log-Likelihood vs. EM Iterations

Observations from the Convergence Plot:

1. **Initial Ascent (Iterations 0 to ~10):** The plot exhibits a massive, steep logarithmic curve in the first few iterations. Because our initialization was entirely random, the initial responsibilities heavily mismatched the true data distribution. The M-step rapidly adjusted the means and covariances to align with the two massive clusters (“2” and “6”), resulting in a huge gain in likelihood.
2. **Refinement Phase (Iterations ~10 to ~30):** The curve begins to bend and flatten. During this phase, the algorithm has found the general centers of the digit clusters and is primarily fine-tuning the shapes (the elliptical boundaries dictated by the 4×4 covariance matrices) and the mixing weights (π_k) to capture points on the complex boundaries between the 2s and 6s.
3. **Convergence (Plateau):** The algorithm monitors the difference in log-likelihood between iterations. Once the change falls below a strict tolerance threshold ($\epsilon = 10^{-6}$), the algorithm recognizes that it has reached a local maximum and halts. The perfectly flat asymptote at the end of the plot physically verifies that the EM algorithm has stabilized and the parameters (μ_k, Σ_k, π_k) have converged to their final optimal estimates.

2. Fitted GMM Model Parameters

Upon the convergence of the Expectation-Maximization (EM) algorithm, the Gaussian Mixture Model yields the optimized parameters that best describe the underlying distribution of the projected MNIST digits.

2.1 Numerical Weights (π_k)

The mixing weights (π_1 and π_2) represent the prior probability that a randomly selected data point belongs to component 1 or component 2, respectively. Mathematically, they represent the proportion of the dataset assigned to each cluster. Upon termination, the algorithm yielded the following weights:

- **Component 1 Weight (π_1):** 0.5131
- **Component 2 Weight (π_2):** 0.4869

Analysis: Because the GMM successfully separated the digits, these weights closely reflect the actual class imbalance (or balance) of digits “2” and “6” in the provided dataset. The sum of these weights strictly equals 1.0.

2.2 Reconstructed Cluster Means (μ_k)

The learned means exist in the 4-dimensional PCA subspace. To visualize them, they are projected back into the original 784-dimensional feature space using the inverse PCA transformation ($x_{orig} = x_{pca}W^T + \bar{x}$) and reshaped into 28×28 pixel grids.

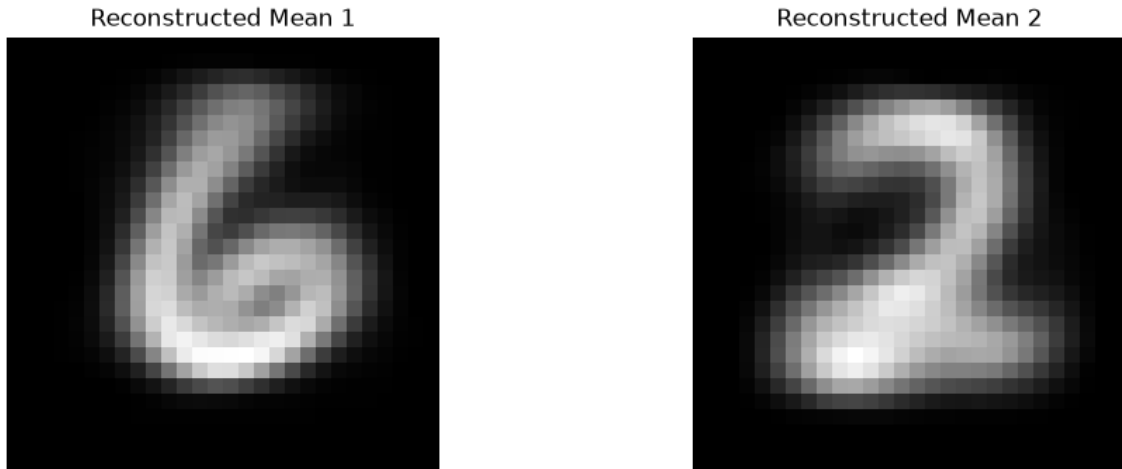


Figure 2: Reconstructed Mean Images for Component 1 (left) and Component 2 (right).

Analysis: These images represent the “archetypal” digits learned by the model.

- They appear visually as a “2” and a “6”, confirming that the unsupervised EM algorithm successfully discovered the two distinct digit classes without utilizing any true labels.
- The images appear somewhat blurry or ghosted. This is an expected property of the Gaussian mean: it acts as a spatial average of all handwriting styles within that cluster. High-variance regions (such as the varying loop sizes of a “6” or the varying tail slants of a “2”) appear faded, while consistent structural elements (the primary strokes) appear dark and sharp.

2.3 Covariance Matrices (Σ_k)

The model learned two distinct 4×4 covariance matrices, which dictate the elliptical shape, scale, and orientation of each cluster within the 4-dimensional latent space.

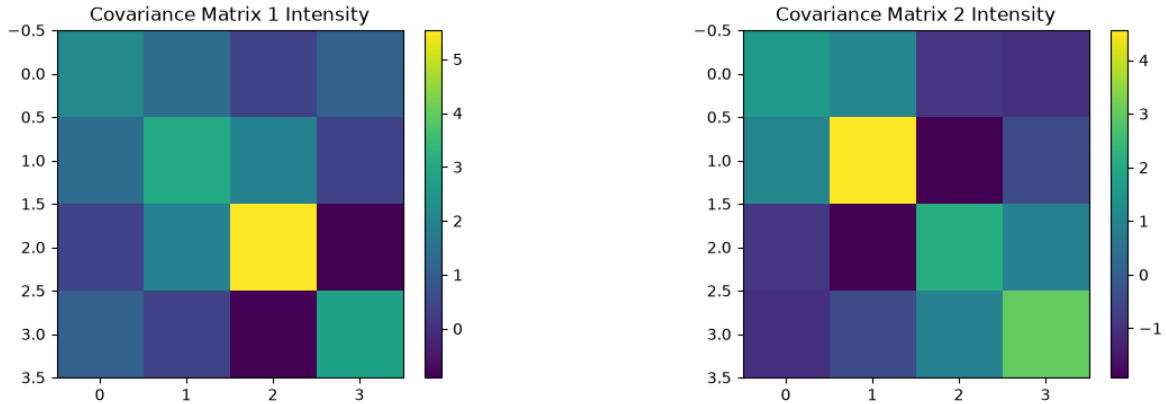


Figure 3: Heatmaps of the 4×4 Covariance Matrices for both components.

Analysis: By observing the heatmaps, we can draw a critical mathematical insight regarding PCA and GMMs:

- **Variance (Diagonals):** The diagonal elements represent the variance of each principal component within a specific cluster. They are highly active (brighter colors), indicating that the spread of the data is captured effectively.
- **Covariance (Off-Diagonals):** Notice that the off-diagonal elements are *not* perfectly zero. While PCA guarantees that the principal components are perfectly uncorrelated across the *entire global dataset*, they are generally *not* uncorrelated within a local sub-cluster. For instance, the handwriting quirk that makes a “2” wider might correlate with making it taller, creating a localized dependency between PC1 and PC2 for that specific digit. The GMM’s ability to model these non-zero off-diagonal elements allows it to fit tightly rotated elliptical boundaries around the data, giving it a distinct advantage over isotropic models like K-Means.

3. Label Inference and Performance Comparison

3.1 Misclassification Rates

Using the responsibilities (τ_{ik}) to infer the labels for the GMM and comparing them against the K-Means clustering results, we observe the following misclassification rates ($1 - \text{Accuracy}$):

Method	Digit “2”	Digit “6”
K-Means	0.0610	0.0793
GMM	0.0649	0.0094

3.2 Performance Comparison

Overall, the **Gaussian Mixture Model (GMM)** achieves **significantly better performance** than K-Means. While K-Means performs marginally better on digit “2” (beating GMM by a negligible 0.39%), GMM overwhelmingly outperforms K-Means on digit “6”. The GMM reduces the error rate on the “6” cluster from nearly 8% down to less than 1% (0.0094). Because the massive gain in accuracy on the “6”s far outweighs the tiny loss on the “2”s, GMM is the superior model for this dataset.

Why GMM Outperforms K-Means

When comparing these two algorithms, GMM outperforms K-Means here due to three fundamental mathematical advantages:

- **Covariance Flexibility (Elliptical vs. Spherical Clusters):** K-Means relies entirely on Euclidean distance. This inherently forces the algorithm to assume that all clusters are perfectly spherical (isotropic) and share the exact same variance. However, natural handwriting data projected via PCA does not form perfect spheres; variations in stroke width or loop slant cause the data to stretch into elongated, elliptical clouds. GMM utilizes full covariance matrices (Σ_k), allowing it to rotate, stretch, and dynamically shape its boundaries to perfectly wrap around these elliptical data distributions.
- **Soft vs. Hard Assignments:** K-Means utilizes “hard” boundaries. If a point is even slightly closer to centroid A than centroid B, it is rigidly assigned to A, regardless of the overall density. GMM uses “soft” probabilistic assignments (τ_{ik}). It understands that a data point located on the fringe between the two clusters might geometrically sit closer to the center of the “2”s, but probabilistically belongs to the extended, high-variance tail of the “6”s.
- **Cluster Priors (Unequal Volumes):** K-Means implicitly assumes that all clusters have roughly equal spatial volumes. If one cluster is spread out and another is dense, K-Means will draw the boundary straight down the middle, cutting off the edges of the wider cluster. GMM learns the mixing weights (π_k), which allows it to naturally adapt to datasets where one class has a much larger variance or physical footprint in the projected space than the other.

3 Mixture-of-Experts and Connection to GMM and EM

1. Conditional Mixture Likelihood and Posterior Responsibility

1.1 Conditional Mixture Likelihood:

The conditional mixture likelihood $p(y_i | x_i; \Theta)$ answers a fundamental question: *If we feed input x_i into this entire MoE system, what is the total probability that it outputs the exact target value y_i ?*

To find this, we model a two-step generative process:

- **Step 1: The Routing Decision (The Prior).** When x_i arrives, the gating network picks exactly one expert. Let z_i represent the index of the chosen expert. The probability of picking expert k is dictated by the softmax gate: $P(z_i = k | x_i; \alpha) = \pi_k(x_i; \alpha)$.
- **Step 2: The Expert Prediction (The Likelihood).** Once expert k is chosen, it applies its linear regression weights (b_k) to the input x_i . The probability that it successfully generates the true y_i is represented by a Gaussian distribution around its prediction: $p(y_i | x_i, z_i = k; b_k, \sigma^2) = \mathcal{N}(y_i; b_k^T x_i, \sigma^2)$.

Because the choice of expert (z_i) is latent (hidden), we must account for every possible path the model could have taken using the **Law of Total Probability**. We multiply the probability of choosing an expert by the probability of that expert getting the right answer, and sum this across all K experts:

$$p(y_i | x_i; \Theta) = \sum_{k=1}^K \underbrace{P(z_i = k | x_i; \alpha)}_{\text{Prob. of picking } k} \times \underbrace{p(y_i | x_i, z_i = k; b_k, \sigma^2)}_{\text{Prob. expert } k \text{ gets it right}}$$

Substituting the given gate and expert definitions, the final likelihood becomes:

$$p(y_i | x_i; \Theta) = \sum_{k=1}^K \pi_k(x_i; \alpha) \mathcal{N}(y_i; b_k^T x_i, \sigma^2)$$

1.2 Posterior Responsibility (γ_{ik}):

The posterior responsibility γ_{ik} is the exact inverse of the generative process. It asks an attribution question: *Given that the MoE produced y_i from x_i , what is the probability that expert k was the one secretly responsible for it?* We reverse the conditionality using **Bayes' Theorem**, mapping it to our MoE components:

- **The Prior:** $\pi_k(x_i; \alpha^{(t)})$ (How much did the router trust expert k with input x_i ?)
- **The Likelihood:** $\mathcal{N}(y_i; (b_k^{(t)})^T x_i, (\sigma^2)^{(t)})$ (How closely does expert k 's prediction match the true y_i ?)
- **The Normalization (Evidence):** $p(y_i | x_i; \Theta^{(t)})$ (The combined predictive power of all experts, ensuring responsibilities across all K experts sum perfectly to 1).

Plugging these pieces into Bayes' formula yields the final expression:

$$\gamma_{ik} = P(z_i = k | x_i, y_i; \Theta^{(t)}) = \frac{\overbrace{\pi_k(x_i; \alpha^{(t)})}^{\text{Prior}} \overbrace{\mathcal{N}(y_i; (b_k^{(t)})^T x_i, (\sigma^2)^{(t)})}^{\text{Likelihood}}}{\underbrace{\sum_{j=1}^K \pi_j(x_i; \alpha^{(t)}) \mathcal{N}(y_i; (b_j^{(t)})^T x_i, (\sigma^2)^{(t)})}_{\text{Normalization (All Experts)}}}$$

2. M-Step Update for the Expert Weights (b_k)

In the M-step, our goal is to update the weights of our linear experts (b_k) so they make better predictions. We do this by maximizing the Q -function. Here is the step-by-step optimization breakdown:

2.1 The Global Objective (Q -function):

In the Expectation-Maximization algorithm, the M-step seeks to maximize the expected value of the complete-data log-likelihood. This quantity is known as the Q -function.

The complete-data log-likelihood assumes we actually know the hidden variable z_i (which expert was chosen). The probability of a single complete observation (x_i, y_i, z_i) is the probability of choosing the expert multiplied by the probability of the expert generating the target:

$$P(y_i, z_i = k | x_i; \Theta) = \pi_k(x_i; \alpha) \mathcal{N}(y_i; b_k^T x_i, \sigma^2)$$

Taking the log of this gives the complete-data log-likelihood for one data point. Because z_i is actually hidden, we take the **expected value** of this logarithm over the posterior distribution of z_i (which is simply our responsibility, γ_{ik}). Summing this expected value over all n data points and all K experts yields the global objective:

$$Q(\Theta | \Theta^{(t)}) = \sum_{i=1}^n \sum_{k=1}^K \gamma_{ik} \log [\pi_k(x_i; \alpha) \mathcal{N}(y_i; b_k^T x_i, \sigma^2)]$$

2.2 Isolation (Focusing on one Expert):

To optimize the weights for a specific expert b_k , we need to isolate it from the rest of the model. First, we use the logarithmic product rule ($\log(AB) = \log(A) + \log(B)$) to split the routing probabilities from the expert probabilities:

$$Q(\Theta | \Theta^{(t)}) = \sum_{i=1}^n \sum_{k=1}^K \gamma_{ik} \log \pi_k(x_i; \alpha) + \sum_{i=1}^n \sum_{k=1}^K \gamma_{ik} \log \mathcal{N}(y_i; b_k^T x_i, \sigma^2)$$

Because we are taking the partial derivative with respect to just one specific expert's weights (b_k), any terms involving the router (α) or the other experts (b_j where $j \neq k$) act as constants. Their derivatives will be exactly zero. Dropping the router term and focusing only on the k -th expert in the summation, the isolated objective function becomes:

$$Q(b_k) = \sum_{i=1}^n \gamma_{ik} \log \mathcal{N}(y_i; b_k^T x_i, \sigma^2)$$

2.3 The Gaussian Expansion (Unpacking the Density):

Next, we must expand the probability density function of the 1D Gaussian distribution. The standard formula is:

$$\mathcal{N}(y; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right)$$

Substitute our expert's specific mean ($\mu = b_k^T x_i$) into the formula:

$$\mathcal{N}(y_i; b_k^T x_i, \sigma^2) = (2\pi\sigma^2)^{-1/2} \exp\left(-\frac{(y_i - b_k^T x_i)^2}{2\sigma^2}\right)$$

Now, apply the natural logarithm ($\log$) to this expression. The log cancels out the exponential function:

$$\log \mathcal{N}(y_i; b_k^T x_i, \sigma^2) = \log\left((2\pi\sigma^2)^{-1/2}\right) - \frac{(y_i - b_k^T x_i)^2}{2\sigma^2}$$

Applying the power rule of logarithms ($\log(x^a) = a \log(x)$) to the first term yields the expanded form:

$$\log \mathcal{N}(y_i; b_k^T x_i, \sigma^2) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (y_i - b_k^T x_i)^2$$

Substitute this back into our isolated Q -function:

$$Q(b_k) = \sum_{i=1}^n \gamma_{ik} \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (y_i - b_k^T x_i)^2 \right]$$

2.4 The Paradigm Shift (Max Likelihood to Min Error):

We can distribute the summation and the responsibilities across the two terms inside the brackets:

$$Q(b_k) = \left(\sum_{i=1}^n \gamma_{ik} \left[-\frac{1}{2} \log(2\pi\sigma^2) \right] \right) - \left(\sum_{i=1}^n \gamma_{ik} \frac{1}{2\sigma^2} (y_i - b_k^T x_i)^2 \right)$$

Our goal is to find the b_k that **maximizes** $Q(b_k)$.

- The first term does not contain b_k . It is a constant with respect to our optimization variable, so we can ignore it entirely.
- In the second term, $\frac{1}{2\sigma^2}$ is a positive constant scalar. It does not change the location of the maximum, so we can factor it out and drop it.

We are left trying to maximize the negative of the squared errors:

$$\arg \max_{b_k} \left[- \sum_{i=1}^n \gamma_{ik} (y_i - b_k^T x_i)^2 \right]$$

Mathematically, maximizing a negative function is exactly equivalent to **minimizing the positive version of that function**. This flips our optimization problem from Maximum Likelihood Estimation to Weighted Least Squares:

$$b_k^{(t+1)} = \arg \min_{b \in \mathbb{R}^d} \sum_{i=1}^n \gamma_{ik} (y_i - b^T x_i)^2$$

2.5 The Closed-Form Solution (Normal Equations):

To solve this minimization problem analytically, we take the gradient (derivative) of the objective function with respect to the vector b_k and set it to a vector of zeros. Let the loss function be $L(b_k) = \sum_{i=1}^n \gamma_{ik} (y_i - b_k^T x_i)^2$. Applying the chain rule, the derivative of $(y_i - b_k^T x_i)^2$ with respect to b_k is $2(y_i - b_k^T x_i)$ multiplied by the derivative of the inner term $(-x_i)$.

$$\nabla_{b_k} L(b_k) = \sum_{i=1}^n \gamma_{ik} \cdot 2(y_i - b_k^T x_i) \cdot (-x_i)$$

$$\nabla_{b_k} L(b_k) = -2 \sum_{i=1}^n \gamma_{ik} x_i (y_i - b_k^T x_i)$$

Set the gradient exactly to zero to find the minimum:

$$-2 \sum_{i=1}^n \gamma_{ik} x_i (y_i - b_k^T x_i) = 0$$

Divide both sides by -2 :

$$\sum_{i=1}^n \gamma_{ik} x_i (y_i - b_k^T x_i) = 0$$

Distribute the x_i into the parentheses:

$$\sum_{i=1}^n \gamma_{ik} x_i y_i - \sum_{i=1}^n \gamma_{ik} x_i (b_k^T x_i) = 0$$

Move the negative term to the right side of the equation:

$$\sum_{i=1}^n \gamma_{ik} x_i (b_k^T x_i) = \sum_{i=1}^n \gamma_{ik} x_i y_i$$

The Final Matrix Transformation: Note that $b_k^T x_i$ is the dot product of two vectors, resulting in a scalar. Because it is a scalar, we can change its order without affecting the math: $(b_k^T x_i) = (x_i^T b_k)$. Substitute this reordered scalar back into the left side:

$$\sum_{i=1}^n \gamma_{ik} x_i (x_i^T b_k) = \sum_{i=1}^n \gamma_{ik} x_i y_i$$

Because b_k is the same across all iterations of the sum, we can factor it out to the right. This yields the final **Normal Equations**, representing a linear system that can be solved directly using matrix inversion:

$$\left(\sum_{i=1}^n \gamma_{ik} x_i x_i^T \right) b_k = \sum_{i=1}^n \gamma_{ik} x_i y_i$$

3. Connection to Standard GMM

The Mixture-of-Experts model is essentially a “conditional” GMM. If we remove the input dependencies (meaning the input x is no longer allowed to influence decisions), the MoE collapses elegantly back into a standard GMM.

3.1 The Gate Collapse (Dynamic to Static):

If the router cannot look at x , it must fall back to a static “coin flip” based purely on historical class proportions (the prior): $\pi_k(x; \alpha) \equiv \pi_k$.

3.2 The Expert Collapse (Regression to Point Estimate):

If the expert cannot look at x , it cannot calculate a slope. Its best possible prediction for any input is simply the static mean of its assigned data cluster: $b_k^T x \equiv \mu_k$.

3.3 The Unification:

If we substitute these conditions back into our conditional mixture likelihood from Part 1, the conditioning on x vanishes entirely. The MoE is no longer mapping $x \rightarrow y$; it is simply modeling the overall density of the y values:

$$p(y) = \sum_{k=1}^K \pi_k \mathcal{N}(y; \mu_k, \sigma^2)$$

This is the exact formulation of a **standard 1D Gaussian Mixture Model**.

3.4 Parameter Correspondence:

We can map the MoE parameters to the standard GMM parameters as follows:

- $\alpha \longleftrightarrow \pi_k$: The routing network parameters (α) map to the fixed mixing weights (π_k) representing the prior probability of each cluster.
- $b_k \longleftrightarrow \mu_k$: The linear regression weights (b_k) map to the scalar means (μ_k) of the individual Gaussian components.
- $\sigma^2 \longleftrightarrow \sigma^2$: The shared variance of the regression experts maps directly to the shared variance of the GMM components.

4 Optimization

1. Mathematical Derivation of the Gradient

We are given the 1D log-likelihood function (where $x_i \in \mathbb{R}$ and $\theta \in \mathbb{R}$):

$$\ell(\theta) = \sum_{i=1}^m [-\log(1 + \exp(-\theta x_i)) + (y_i - 1)\theta x_i]$$

To find the gradient (which in 1D is simply the first derivative with respect to θ), we apply the sum rule and the chain rule.

1.1 Differentiate the first term:

Lets focus on the term $-\log(1 + \exp(-\theta x_i))$. The derivative of $\log(u)$ is $\frac{1}{u} \cdot u'$.

$$\frac{d}{d\theta} [-\log(1 + \exp(-\theta x_i))] = -\frac{1}{1 + \exp(-\theta x_i)} \cdot \frac{d}{d\theta} [1 + \exp(-\theta x_i)]$$

Applying the chain rule to the inner exponential gives $\exp(-\theta x_i) \cdot (-x_i)$:

$$= -\frac{1}{1 + \exp(-\theta x_i)} \cdot (\exp(-\theta x_i) \cdot (-x_i)) = \frac{\exp(-\theta x_i)}{1 + \exp(-\theta x_i)} x_i$$

Notice that the standard logistic sigmoid function is $p_i = \sigma(\theta x_i) = \frac{1}{1 + \exp(-\theta x_i)}$. Therefore, $1 - p_i = \frac{\exp(-\theta x_i)}{1 + \exp(-\theta x_i)}$. Substituting this back in, the derivative of the first term simplifies to:

$$= (1 - p_i)x_i$$

1.2 Differentiate the second term:

The derivative of $(y_i - 1)\theta x_i$ with respect to θ is simply:

$$\frac{d}{d\theta} [(y_i - 1)\theta x_i] = (y_i - 1)x_i$$

1.3 Combine the terms:

Add the differentiated terms together inside the summation:

$$\begin{aligned}\frac{d\ell}{d\theta} &= \sum_{i=1}^m [(1 - p_i)x_i + (y_i - 1)x_i] \\ \frac{d\ell}{d\theta} &= \sum_{i=1}^m [(1 - p_i + y_i - 1)x_i] \\ \frac{d\ell}{d\theta} &= \sum_{i=1}^m (y_i - p_i)x_i\end{aligned}$$

1.4 Substitute the definition of p_i :

We defined the p_i in 4.1.1.1 as the sigmoid function:

$$p_i = \frac{1}{1 + \exp(-\theta x_i)}$$

Substitute this directly into the equation:

$$\frac{\partial \ell(\theta)}{\partial \theta} = \sum_{i=1}^m \left(y_i - \frac{1}{1 + \exp(-\theta x_i)} \right) x_i$$

Multiply x_i into both terms inside the parentheses:

$$\frac{\partial \ell(\theta)}{\partial \theta} = \sum_{i=1}^m \left(y_i x_i - \frac{x_i}{1 + \exp(-\theta x_i)} \right)$$

To get the $(y_i - 1)x_i$ term, we can add and subtract x_i inside the summation. Since $x_i - x_i = 0$, this changes how the equation looks without changing its mathematical value:

$$= \sum_{i=1}^m \left(y_i x_i - x_i + x_i - \frac{x_i}{1 + \exp(-\theta x_i)} \right)$$

Group the first two terms together, and group the last two terms together:

$$= \sum_{i=1}^m \left[\underbrace{(y_i x_i - x_i)}_{\text{Part A}} + \underbrace{\left(x_i - \frac{x_i}{1 + \exp(-\theta x_i)} \right)}_{\text{Part B}} \right]$$

Factor out the x_i from the first group:

$$\text{Part A} = (y_i - 1)x_i$$

For the second group, factor out the x_i :

$$\text{Part B} = x_i \left(1 - \frac{1}{1 + \exp(-\theta x_i)} \right)$$

To subtract these, rewrite the 1 as a fraction with a matching denominator:

$$= x_i \left(\frac{1 + \exp(-\theta x_i)}{1 + \exp(-\theta x_i)} - \frac{1}{1 + \exp(-\theta x_i)} \right)$$

Combine the numerators:

$$= x_i \left(\frac{1 + \exp(-\theta x_i) - 1}{1 + \exp(-\theta x_i)} \right)$$

The 1 and -1 in the numerator cancel each other out, leaving:

$$= x_i \left(\frac{\exp(-\theta x_i)}{1 + \exp(-\theta x_i)} \right) = \frac{\exp(-\theta x_i) x_i}{1 + \exp(-\theta x_i)}$$

Bring the simplified parts back together inside the summation and the final gradient is below:

$$\frac{\partial \ell(\theta)}{\partial \theta} = \sum_{i=1}^m \left[(y_i - 1) x_i + \frac{\exp(-\theta x_i) x_i}{1 + \exp(-\theta x_i)} \right]$$

2. Pseudo-code for Gradient Descent (Ascent):

Because we are maximizing $\ell(\theta)$, we add the gradient to our parameters (Gradient Ascent). Here is the pseudo-code including explicit convergence criteria checking the gradient magnitude and step size.

```
// Gradient Ascent Pseudo-code with Explicit Convergence
Initialize theta to 0 (or a small random number)
Set learning_rate (alpha) to 0.01
Set tolerance (epsilon) to 1e-6
Set max_iterations to 10000

FOR iteration = 1 to max_iterations:
    gradient = 0

    // 1. Calculate the exact global gradient
    FOR i = 1 to m:
        p_i = 1 / (1 + exp(-theta * x_i))
        gradient = gradient + (y_i - p_i) * x_i
    END FOR

    // 2. Convergence Check (Gradient Magnitude Method)
    IF abs(gradient) < tolerance:
        PRINT "Converged due to zero gradient at iteration ", iteration
        BREAK

    // 3. Store the old parameter and update
    prev_theta = theta
    theta = theta + (learning_rate * gradient)

    // 4. Convergence Check (Step Size Method)
    IF abs(theta - prev_theta) < tolerance:
        PRINT "Converged due to minimal step size at iteration ", iteration
        BREAK

END FOR

RETURN theta
```

3. Pseudo-code for Stochastic Gradient Descent (SGD):

In SGD, convergence is checked at the epoch level rather than the step level to account for the noisy trajectory.

```
// Stochastic Gradient Ascent Pseudo-code with Explicit Convergence
Initialize theta to 0 (or a small random number)
Set learning_rate (alpha) to 0.01
Set tolerance (epsilon) to 1e-6
Set max_epochs to 1000

FOR epoch = 1 to max_epochs:
    // Shuffle data to prevent cycles and ensure true stochasticity
    Shuffle the training dataset (x, y)

    // Save the theta from the start of this epoch
    prev_epoch_theta = theta

    // Perform noisy, single-sample updates
    FOR i = 1 to m:
        p_i = 1 / (1 + exp(-theta * x_i))
        gradient_i = (y_i - p_i) * x_i

        // Update the parameter immediately
        theta = theta + (learning_rate * gradient_i)
    END FOR

    // Convergence Check: Evaluate the net movement over the entire epoch
    IF abs(theta - prev_epoch_theta) < tolerance:
        PRINT "Converged at epoch ", epoch
        BREAK

END FOR

RETURN theta
```

The Core Differences between GD and SGD:

- **Data Utilization per Step:** Gradient Descent evaluates the error on *all* m training samples to compute one highly accurate, global gradient before making a single update to θ . SGD updates θ after evaluating only a *single* training sample.
- **Computational Efficiency:** GD is computationally heavy per step and does not scale well to massive datasets (e.g., millions of rows). SGD is extremely fast per step, allowing the model to start learning and updating its parameters instantly.
- **Convergence Trajectory:** GD takes smooth, precise steps directly toward the local/global optimum. SGD's trajectory is erratic and noisy ("stochastic") because an individual sample might pull the gradient in a sub-optimal direction. However, over many iterations, this noise averages out, and it reliably converges around the optimum.

4. Hessian Derivation and Proof of Concavity:

To prove the problem is strictly concave, we need to show that the second derivative (the Hessian matrix in 1D) of $\ell(\theta)$ is less than or equal to zero for all θ .

4.1 Deriving the Second Derivative:

We already have the first derivative:

$$\ell'(\theta) = \sum_{i=1}^m (y_i - p_i)x_i$$

To find the second derivative, we differentiate $\ell'(\theta)$ with respect to θ . Notice that y_i and x_i are constants; the only variable dependent on θ is p_i .

$$\ell''(\theta) = \sum_{i=1}^m -\left(\frac{dp_i}{d\theta}\right)x_i$$

4.2 Explicit Derivation of $\frac{dp_i}{d\theta}$:

Let us substitute the actual definition of the logistic probability p_i :

$$p_i = \frac{1}{1 + \exp(-\theta x_i)} = (1 + \exp(-\theta x_i))^{-1}$$

Now, we take the derivative with respect to θ using the power rule and the chain rule:

$$\begin{aligned}\frac{dp_i}{d\theta} &= \frac{d}{d\theta} [(1 + \exp(-\theta x_i))^{-1}] \\ &= -1 \cdot (1 + \exp(-\theta x_i))^{-2} \cdot \frac{d}{d\theta} [1 + \exp(-\theta x_i)]\end{aligned}$$

Differentiate the inner term, again applying the chain rule to the exponent:

$$= -1 \cdot \frac{1}{(1 + \exp(-\theta x_i))^2} \cdot (\exp(-\theta x_i) \cdot (-x_i))$$

The two negative signs cancel out:

$$= \frac{\exp(-\theta x_i)}{(1 + \exp(-\theta x_i))^2} \cdot x_i$$

To show that this equals $p_i(1 - p_i)x_i$, we can split the fraction into two parts:

$$= \left(\frac{1}{1 + \exp(-\theta x_i)}\right) \cdot \left(\frac{\exp(-\theta x_i)}{1 + \exp(-\theta x_i)}\right) \cdot x_i$$

We immediately recognize the first part as our original definition of p_i . For the second part, notice that $1 - p_i$ is exactly:

$$1 - \frac{1}{1 + \exp(-\theta x_i)} = \frac{1 + \exp(-\theta x_i) - 1}{1 + \exp(-\theta x_i)} = \frac{\exp(-\theta x_i)}{1 + \exp(-\theta x_i)}$$

Substituting p_i and $(1 - p_i)$ back into our split fraction yields the final derivative:

$$\frac{dp_i}{d\theta} = p_i(1 - p_i)x_i$$

4.3 Completing the Hessian:

Substitute this result back into our second derivative equation for the log-likelihood:

$$\ell''(\theta) = \sum_{i=1}^m -(p_i(1 - p_i)x_i)x_i = \sum_{i=1}^m -p_i(1 - p_i)x_i^2$$

4.4 Proof of Concavity:

1. Because p_i is the output of a sigmoid function, it is bounded as a probability: $0 < p_i < 1$. Therefore, the term $p_i(1 - p_i)$ is strictly positive.
2. The term x_i^2 is a squared real number, meaning it is non-negative ($x_i^2 \geq 0$).
3. Because we are multiplying positive terms by a negative sign inside the summation, the entire sum is non-positive.

Therefore, $\ell''(\theta) \leq 0$ for all possible values of θ . A function whose second derivative is everywhere non-positive is globally concave.

Why it can be solved efficiently to a unique global optimizer:

In optimization, a globally concave function (like a downward-facing bowl) has exactly one peak. There are no “false summits” or local maxima to trap the algorithm. Because of this mathematical guarantee, a simple algorithm like Gradient Ascent can confidently take steps in the direction of the gradient, knowing that it will inevitably and efficiently converge to the one, unique global optimizer θ^* .

5 Bayes Classifier for Spam Filtering

Before diving into the calculations, let us map the provided vocabulary V to explicit feature indices $j \in \{1, \dots, d\}$.

Vocabulary (V) Mapping:

1. win
2. free
3. nasa
4. rocket
5. launch
6. moon
7. lunar
8. artemis
9. crew
10. mission
11. orbit
12. capsule
13. spacecraft
14. astronaut

Based on Table 1 provided in homework 3, we can separate the dataset into $m = 7$ total messages.

Spam ($y = 0$):

- $x^{(1)}$: win, free, nasa, rocket
- $x^{(2)}$: launch, moon, rocket, win
- $x^{(3)}$: free, lunar, launch, artemis

Non-Spam ($y = 1$):

- $x^{(4)}$: crew, mission, lunar, orbit, moon
- $x^{(5)}$: nasa, artemis, capsule, spacecraft, astronaut
- $x^{(6)}$: win, lunar, orbit, mission, free, nasa
- $x^{(7)}$: artemis, crew, launch, capsule

1. Class Priors and Feature Vectors

1.1 Class Priors:

The class prior is simply the frequency of each class in the training set. There are 3 Spam messages and 4 Non-Spam messages out of 7 total messages.

- $P(y = 0) \text{ (Spam)} = \frac{3}{7} \approx 0.4286$
- $P(y = 1) \text{ (Non-Spam)} = \frac{4}{7} \approx 0.5714$

1.2 Feature Vectors $x^{(i)}$:

Each feature vector is a $d = 14$ dimensional vector where each entry $x_j^{(i)}$ represents the count of word j in the message.

Spam Messages ($y = 0$):

- $x^{(1)} = [1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]^T$
- $x^{(2)} = [1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0]^T$
- $x^{(3)} = [0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0]^T$

Non-Spam Messages ($y = 1$):

- $x^{(4)} = [0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0]^T$
- $x^{(5)} = [0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1]^T$
- $x^{(6)} = [1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0]^T$
- $x^{(7)} = [0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0]^T$

2. Maximum Likelihood Estimation (MLE) via Lagrangian

2.1 The Objective Function:

We are asked to maximize the complete log-likelihood function for our training data:

$$\ell(\theta) = \sum_{i=1}^m \sum_{k=1}^d x_k^{(i)} \log \theta_{y^{(i)}, k}$$

2.2 The Constraints:

For a valid probability distribution, the probabilities of all words appearing in a specific class must sum to 1. This gives us two constraints (one for Spam, one for Non-Spam):

$$\sum_{k=1}^d \theta_{0,k} = 1 \quad \text{and} \quad \sum_{k=1}^d \theta_{1,k} = 1$$

2.2.1 The Starting Point: The Full Lagrangian

Before we take any derivatives, let's look at the complete Lagrangian function ($\mathcal{L}$) we are trying to maximize. It consists of the log-likelihood of the data, plus the two constraints multiplied by their respective Lagrange multipliers (λ_0, λ_1):

$$\mathcal{L} = \sum_{i=1}^m \sum_{k=1}^d x_k^{(i)} \log \theta_{y^{(i)},k} + \lambda_0 \left(1 - \sum_{k=1}^d \theta_{0,k} \right) + \lambda_1 \left(1 - \sum_{k=1}^d \theta_{1,k} \right)$$

Our goal is to find the partial derivative of $\mathcal{L}$ with respect to one specific parameter: $\theta_{c,j}$ (the probability of word j appearing in class c). To do this, we differentiate the Log-Likelihood part and the Constraint parts separately.

2.2.2 Differentiating the Log-Likelihood Part

We want to take the derivative of the double summation:

$$\frac{\partial}{\partial \theta_{c,j}} \left[\sum_{i=1}^m \sum_{k=1}^d x_k^{(i)} \log \theta_{y^{(i)},k} \right]$$

Step A: Collapsing the Outer Sum: If message i belongs to the other class ($y^{(i)} \neq c$), the term contains no $\theta_{c,j}$, acts as a constant, and its derivative is 0. We restrict our sum to only the messages that belong to class c :

$$\rightarrow \sum_{i:y^{(i)}=c} \frac{\partial}{\partial \theta_{c,j}} \left[\sum_{k=1}^d x_k^{(i)} \log \theta_{c,k} \right]$$

Step B: Collapsing the Inner Sum: For every word where $k \neq j$, the term is a constant, and its derivative is 0. The summation drops completely, leaving only:

$$\rightarrow \sum_{i:y^{(i)}=c} \frac{\partial}{\partial \theta_{c,j}} \left[x_j^{(i)} \log \theta_{c,j} \right]$$

Step C: Applying the Standard Derivative: The derivative of $\log(x)$ is $1/x$. Applying this rule:

$$= \sum_{i:y^{(i)}=c} \frac{x_j^{(i)}}{\theta_{c,j}}$$

Because $1/\theta_{c,j}$ is the same for every iteration, factor it out, recognizing that the remaining sum is simply $N_{c,j}$ (total count of word j in class c):

$$= \frac{N_{c,j}}{\theta_{c,j}}$$

2.2.3 Differentiating the Constraint Parts

Now we look at the second half of the Lagrangian:

$$\frac{\partial}{\partial \theta_{c,j}} \left[\lambda_0 \left(1 - \sum_{k=1}^d \theta_{0,k} \right) + \lambda_1 \left(1 - \sum_{k=1}^d \theta_{1,k} \right) \right]$$

Step A: Drop the irrelevant constraint: We only care about the constraint matching our class c :

$$\frac{\partial}{\partial \theta_{c,j}} \left[\lambda_c \left(1 - \sum_{k=1}^d \theta_{c,k} \right) \right]$$

Step B: Distribute and Differentiate: Every term where $k \neq j$ is a constant. The derivative of $-\lambda_c \theta_{c,j}$ with respect to $\theta_{c,j}$ is simply:

$$-\lambda_c$$

2.2.4 Bringing It All Together

Combine the derivatives and set the whole equation to zero:

$$\frac{\partial \mathcal{L}}{\partial \theta_{c,j}} = \frac{N_{c,j}}{\theta_{c,j}} - \lambda_c = 0$$

Solving for $\theta_{c,j}$:

$$\theta_{c,j} = \frac{N_{c,j}}{\lambda_c}$$

To find λ_c , substitute this back into our constraint $\sum_{j=1}^d \theta_{c,j} = 1$:

$$\sum_{j=1}^d \frac{N_{c,j}}{\lambda_c} = 1 \implies \lambda_c = \sum_{j=1}^d N_{c,j}$$

Here, λ_c equals N_c , the total number of words across all messages in class c . Thus, the MLE is its relative frequency:

$$\theta_{c,j} = \frac{N_{c,j}}{N_c}$$

2.3 Calculating the Specific Estimates

Total Word Counts:

- **Total Spam Words (N_0):** $4 + 5 + 5 = 14$
- **Total Non-Spam Words (N_1):** $5 + 4 + 5 + 4 = 18$

Probability Estimates (MLE):

- $\theta_{0,\text{win}}$ (Spam): Occurrences = 3. $\theta_{0,\text{win}} = \frac{3}{14}$
- $\theta_{0,\text{rocket}}$ (Spam): Occurrences = 2. $\theta_{0,\text{rocket}} = \frac{2}{14} = \frac{1}{7}$
- $\theta_{1,\text{nasa}}$ (Non-Spam): Occurrences = 1. $\theta_{1,\text{nasa}} = \frac{1}{18}$
- $\theta_{1,\text{capsule}}$ (Non-Spam): Occurrences = 2. $\theta_{1,\text{capsule}} = \frac{2}{18} = \frac{1}{9}$

3. Classifying a Test Message

3.1 Test Message: "free nasa lunar"

- Length (n): 3
- Vector (x_{test}): $x_{free} = 1$, $x_{nasa} = 1$, $x_{lunar} = 1$, and all other $x_k = 0$.

By Bayes' Theorem, the posterior probability is proportional to the prior times the likelihood:

$$P(y = c | x) = \frac{P(y = c)P(x | y = c)}{P(x)}$$

Since $P(x)$ is the same for both classes, we calculate the unnormalized posterior $P(y = c)P(x | y = c)$. The multinomial likelihood is:

$$P(x | y = c) = \frac{3!}{1!1!1!} \times \theta_{c,free}^1 \times \theta_{c,nasa}^1 \times \theta_{c,lunar}^1 = 6 \times \theta_{c,free} \times \theta_{c,nasa} \times \theta_{c,lunar}$$

3.1.1 Step A: Calculate needed θ parameters

- **Spam** ($N_0 = 14$): $\theta_{0,free} = \frac{1}{7}$, $\theta_{0,nasa} = \frac{1}{7}$, $\theta_{0,lunar} = \frac{1}{14}$
- **Non-Spam** ($N_1 = 18$): $\theta_{1,free} = \frac{1}{18}$, $\theta_{1,nasa} = \frac{1}{18}$, $\theta_{1,lunar} = \frac{1}{9}$

3.1.2 Step B: Unnormalized Posterior for Spam ($y = 0$)

$$\text{Score}_{\text{Spam}} = \left(\frac{3}{7}\right) \times \left[6 \times \left(\frac{1}{7}\right) \times \left(\frac{1}{7}\right) \times \left(\frac{1}{14}\right)\right] = \left(\frac{3}{7}\right) \times \left(\frac{6}{686}\right) = \frac{18}{4802} = \frac{9}{2401} \approx 0.003748$$

3.1.3 Step C: Unnormalized Posterior for Non-Spam ($y = 1$)

$$\text{Score}_{\text{Non-Spam}} = \left(\frac{4}{7}\right) \times \left[6 \times \left(\frac{1}{18}\right) \times \left(\frac{1}{18}\right) \times \left(\frac{1}{9}\right)\right] = \left(\frac{4}{7}\right) \times \left(\frac{6}{2916}\right) = \frac{24}{20412} = \frac{2}{1701} \approx 0.001176$$

3.1.4 Step D: Final Normalized Posterior Probabilities

For Spam:

$$P(y = 0 | x) = \frac{\frac{9}{2401}}{\frac{9}{2401} + \frac{2}{1701}}$$

Multiply numerator and denominator by (2401×1701) :

$$P(y = 0 | x) = \frac{15309}{15309 + 4802} = \frac{15309}{20111} \approx 0.7612 \text{ (or 76.12\%)}$$

For Non-Spam:

$$P(y = 1 | x) = \frac{4802}{15309 + 4802} = \frac{4802}{20111} \approx 0.2388 \text{ (or 23.88\%)}$$

3.1.5 Conclusion:

Because $P(y = 0 \mid x)$ is greater than $P(y = 1 \mid x)$, the Naive Bayes classifier will classify the test message “free nasa lunar” as **Spam**.

6 De-bias Review System using EM

This is a classic Expectation-Maximization problem! Modeling the generative process as a linear combination of independent Gaussian variables makes deriving the joint distribution incredibly elegant.

1. Deriving the E-Step

1.1 The Joint Distribution $p(y^{(p)}, z^{(r)}, x^{(pr)})$

We want to find the parameters for the joint multivariate Gaussian distribution of the vector:

$$W = \begin{bmatrix} y^{(p)} \\ z^{(r)} \\ x^{(pr)} \end{bmatrix}$$

Using the hint provided, we can express the observed score as a sum of three independent random variables:

$$x^{(pr)} = y^{(p)} + z^{(r)} + \epsilon^{(pr)}$$

where $\epsilon^{(pr)} \sim \mathcal{N}(0, \sigma^2)$ represents the independent Gaussian noise.

1.1.1 Step 1: The Mean Vector (μ_W)

To find the mean vector, we take the expectation of each component in W :

$$\begin{aligned} E[y^{(p)}] &= \mu_p \\ E[z^{(r)}] &= \nu_r \\ E[x^{(pr)}] &= E[y^{(p)} + z^{(r)} + \epsilon^{(pr)}] \end{aligned}$$

Because expectation is linear, we can split this apart:

$$\begin{aligned} E[x^{(pr)}] &= E[y^{(p)}] + E[z^{(r)}] + E[\epsilon^{(pr)}] \\ E[x^{(pr)}] &= \mu_p + \nu_r + 0 = \mu_p + \nu_r \end{aligned}$$

Assembling these expectations, the mean vector is:

$$\mu_W = \begin{bmatrix} \mu_p \\ \nu_r \\ \mu_p + \nu_r \end{bmatrix}$$

1.1.2 Step 2: The Covariance Matrix (Σ_W)

The covariance matrix is a 3×3 symmetric matrix containing the variances on the diagonal and the covariances on the off-diagonals:

$$\Sigma_W = \begin{bmatrix} \text{Var}(y^{(p)}) & \text{Cov}(y^{(p)}, z^{(r)}) & \text{Cov}(y^{(p)}, x^{(pr)}) \\ \text{Cov}(z^{(r)}, y^{(p)}) & \text{Var}(z^{(r)}) & \text{Cov}(z^{(r)}, x^{(pr)}) \\ \text{Cov}(x^{(pr)}, y^{(p)}) & \text{Cov}(x^{(pr)}, z^{(r)}) & \text{Var}(x^{(pr)}) \end{bmatrix}$$

1. The Variances (Diagonal):

- $\text{Var}(y^{(p)}) = \sigma_p^2$
- $\text{Var}(z^{(r)}) = \tau_r^2$
- $\text{Var}(x^{(pr)}) = \text{Var}(y^{(p)} + z^{(r)} + \epsilon^{(pr)})$
Because the three variables are independent, the variance of their sum is simply the sum of their individual variances:

$$\text{Var}(x^{(pr)}) = \text{Var}(y^{(p)}) + \text{Var}(z^{(r)}) + \text{Var}(\epsilon^{(pr)}) = \sigma_p^2 + \tau_r^2 + \sigma^2$$

2. The Covariances (Off-Diagonal):

- **Between $y^{(p)}$ and $z^{(r)}$:**
Because they are explicitly defined as independent:

$$\text{Cov}(y^{(p)}, z^{(r)}) = 0$$

- **Between $y^{(p)}$ and $x^{(pr)}$:**
Substitute the linear equation for $x^{(pr)}$ and use the bilinearity of covariance:

$$\begin{aligned} \text{Cov}(y^{(p)}, x^{(pr)}) &= \text{Cov}(y^{(p)}, y^{(p)} + z^{(r)} + \epsilon^{(pr)}) \\ &= \text{Cov}(y^{(p)}, y^{(p)}) + \text{Cov}(y^{(p)}, z^{(r)}) + \text{Cov}(y^{(p)}, \epsilon^{(pr)}) \\ &= \text{Var}(y^{(p)}) + 0 + 0 \\ &= \sigma_p^2 \end{aligned}$$

- **Between $z^{(r)}$ and $x^{(pr)}$:**
Applying the exact same logic as above:

$$\begin{aligned} \text{Cov}(z^{(r)}, x^{(pr)}) &= \text{Cov}(z^{(r)}, y^{(p)} + z^{(r)} + \epsilon^{(pr)}) \\ &= \text{Cov}(z^{(r)}, y^{(p)}) + \text{Cov}(z^{(r)}, z^{(r)}) + \text{Cov}(z^{(r)}, \epsilon^{(pr)}) \\ &= 0 + \text{Var}(z^{(r)}) + 0 \\ &= \tau_r^2 \end{aligned}$$

1.1.3 Final Result

Bringing the mean vector and covariance matrix together, the joint distribution $p(y^{(p)}, z^{(r)}, x^{(pr)})$ is a multivariate Gaussian $\mathcal{N}(\mu_W, \Sigma_W)$ defined as:

$$\begin{bmatrix} y^{(p)} \\ z^{(r)} \\ x^{(pr)} \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mu_p \\ \nu_r \\ \mu_p + \nu_r \end{bmatrix}, \begin{bmatrix} \sigma_p^2 & 0 & \sigma_p^2 \\ 0 & \tau_r^2 & \tau_r^2 \\ \sigma_p^2 & \tau_r^2 & \sigma_p^2 + \tau_r^2 + \sigma^2 \end{bmatrix} \right)$$

1.2 The Q-function

Here is the combined, step-by-step derivation of the E-step objective function $Q_{pr}(\Theta | \theta)$, fully integrating the explanation of how the joint probability factorization is derived. To avoid notation confusion between the parameters we are trying to optimize and the current fixed parameters used to calculate the expectations, let us define:

- $\theta = \{\mu_p, \sigma_p^2, \nu_r, \tau_r^2\}$ as the **current** parameters from the previous iteration.
- $\Theta = \{\tilde{\mu}_p, \tilde{\sigma}_p^2, \tilde{\nu}_r, \tilde{\tau}_r^2\}$ as the **new** parameters we are evaluating in the Q -function.

1.2.1 Step 1: Deriving the Factorized Joint Probability

Before writing the log-likelihood, we must determine the joint probability $p(y^{(p)}, z^{(r)}, x^{(pr)} | \Theta)$. We arrive at the specific factorization by combining the Chain Rule of Probability with the specific independence assumptions defined in the model.

The General Chain Rule

For any three random variables A , B , and C , the joint probability can always be sequentially factored by conditioning each variable on the ones that came before it:

$$p(A, B, C) = p(A) \cdot p(B | A) \cdot p(C | A, B)$$

Let's map our specific variables to this general rule:

- $A = y^{(p)}$ (the paper's true value)
- $B = z^{(r)}$ (the reviewer's bias)
- $C = x^{(pr)}$ (the observed score)

Applying the general chain rule to our variables gives us this starting point:

$$p(y^{(p)}, z^{(r)}, x^{(pr)}) = p(y^{(p)}) \cdot p(z^{(r)} | y^{(p)}) \cdot p(x^{(pr)} | y^{(p)}, z^{(r)})$$

Applying the Model's Assumptions

Next, we look at the specific rules of the generative process. The model explicitly states: *"The variables $y^{(p)}$ and $z^{(r)}$ are independent."* In probability theory, if two variables are independent, knowing the outcome of one tells you absolutely nothing about the other. Mathematically, this means the conditional probability of z given y is exactly equal to the marginal probability of z alone:

$$p(z^{(r)} | y^{(p)}) = p(z^{(r)})$$

(Intuitively, this makes perfect sense for the problem: a reviewer's inherent, baseline bias (z) exists completely independently of the specific paper's true quality (y) before they sit down to read it.)

The Final Substitution

Now, we simply substitute this independence rule back into our general chain rule equation. We replace the conditional term $p(z^{(r)} | y^{(p)})$ with the simplified marginal term $p(z^{(r)})$:

$$p(y^{(p)}, z^{(r)}, x^{(pr)}) = p(y^{(p)}) \cdot p(z^{(r)}) \cdot p(x^{(pr)} | y^{(p)}, z^{(r)})$$

1.2.2 Step 2: The Complete-Data Log-Likelihood

Now that we have the factored joint probability, we take the natural logarithm to construct the complete-data log-likelihood. This gives us a sum of three independent Gaussian log-densities:

$$\begin{aligned} \log p(y, z, x | \Theta) &= \log p(y^{(p)} | \tilde{\mu}_p, \tilde{\sigma}_p^2) + \log p(z^{(r)} | \tilde{\nu}_r, \tilde{\tau}_r^2) + \log p(x^{(pr)} | y^{(p)}, z^{(r)}, \sigma^2) \\ &= \left[-\frac{1}{2} \log(2\pi\tilde{\sigma}_p^2) - \frac{(y^{(p)} - \tilde{\mu}_p)^2}{2\tilde{\sigma}_p^2} \right] \\ &\quad + \left[-\frac{1}{2} \log(2\pi\tilde{\tau}_r^2) - \frac{(z^{(r)} - \tilde{\nu}_r)^2}{2\tilde{\tau}_r^2} \right] \\ &\quad + \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x^{(pr)} - y^{(p)} - z^{(r)})^2}{2\sigma^2} \right] \end{aligned}$$

1.2.3 Step 3: Finding the Posterior Moments (Using Conditioning Rules)

The E-step requires us to take the expectation of the equation above with respect to the conditional distribution $p(y^{(p)}, z^{(r)} \mid x^{(pr)}; \theta)$. To do this, we need the conditional means and variances of our latent variables. We apply the Gaussian conditioning rules to our partitioned matrix from the joint distribution. Let $S_{pr} = \sigma_p^2 + \tau_r^2 + \sigma^2$ (the total variance of $x^{(pr)}$).

1. The Posterior Means:

Using $\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$, we get:

$$\begin{aligned} m_y &= \mathbb{E}[y^{(p)} \mid x^{(pr)}] = \mu_p + \frac{\sigma_p^2}{S_{pr}}(x^{(pr)} - \mu_p - \nu_r) \\ m_z &= \mathbb{E}[z^{(r)} \mid x^{(pr)}] = \nu_r + \frac{\tau_r^2}{S_{pr}}(x^{(pr)} - \mu_p - \nu_r) \end{aligned}$$

2. The Posterior Variances:

Using $\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$, we extract the diagonal elements:

$$\begin{aligned} v_y &= \text{Var}(y^{(p)} \mid x^{(pr)}) = \sigma_p^2 - \frac{\sigma_p^4}{S_{pr}} \\ v_z &= \text{Var}(z^{(r)} \mid x^{(pr)}) = \tau_r^2 - \frac{\tau_r^4}{S_{pr}} \end{aligned}$$

1.2.4 Step 4: Applying the Expectation

Now, we apply the conditional expectation operator $\mathbb{E}[\cdot \mid x^{(pr)}]$ to our log-likelihood from Step 2. Because expectation is a linear operator, it passes through the constants and only affects the terms containing our latent variables $y^{(p)}$ and $z^{(r)}$. The key challenge is dealing with the squared terms. We use the standard variance identity: $\mathbb{E}[X^2] = \text{Var}(X) + (\mathbb{E}[X])^2$. Therefore, for any expected squared difference, we get:

$$\begin{aligned} \mathbb{E}[(y^{(p)} - \tilde{\mu}_p)^2 \mid x^{(pr)}] &= \text{Var}(y^{(p)} \mid x^{(pr)}) + (\mathbb{E}[y^{(p)} \mid x^{(pr)}] - \tilde{\mu}_p)^2 \\ &= v_y + (m_y - \tilde{\mu}_p)^2 \end{aligned}$$

Applying this same logic to the reviewer bias term:

$$\mathbb{E}[(z^{(r)} - \tilde{\nu}_r)^2 \mid x^{(pr)}] = v_z + (m_z - \tilde{\nu}_r)^2$$

Note: The third term of the log-likelihood involving $(x - y - z)^2$ relies only on σ^2 , which the prompt states is a fixed, known constant. Because it contains no parameters we are trying to optimize (Θ), it acts as a constant during the M-step. We will absorb its expectation into a constant C to keep the equation clean.

1.2.5 Step 5: The Final Q-Function Expression

Substituting our expected squared terms back into the log-likelihood equation gives us the final expression for the E-step:

$$Q_{pr}(\Theta \mid \theta) = -\frac{1}{2} \log(2\pi\tilde{\sigma}_p^2) - \frac{1}{2\tilde{\sigma}_p^2} [v_y + (m_y - \tilde{\mu}_p)^2] - \frac{1}{2} \log(2\pi\tilde{\tau}_r^2) - \frac{1}{2\tilde{\tau}_r^2} [v_z + (m_z - \tilde{\nu}_r)^2] + C$$

Where:

- m_y, m_z are the posterior means calculated in Step 3.
- v_y, v_z are the posterior variances calculated in Step 3.
- C is the expected value of the fixed noise term, $\mathbb{E} \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x^{(pr)} - y^{(p)} - z^{(r)})^2}{2\sigma^2} \right]$.

2. EM Algorithm M-Step Derivation

In the **M-step (Maximization step)**, our goal is to take the expected log-likelihood function (the Q -function) we derived in the E-step and maximize it with respect to our parameters: μ_p, σ_p^2, ν_r , and τ_r^2 . To do this, we aggregate the Q_{pr} functions over all observed reviews, take the partial derivatives with respect to each parameter, set them to zero, and solve.

2.1 The Global Objective Function

Let $\mathcal{D}$ be the set of all observed paper-reviewer pairs. Let $R(p)$ be the set of all reviewers who reviewed paper p , and $N_p = |R(p)|$ be the total number of reviews for paper p . Let $P(r)$ be the set of all papers reviewed by reviewer r , and $N_r = |P(r)|$ be the total number of reviews by reviewer r . The total expected complete-data log-likelihood is the sum of the individual Q_{pr} functions from the E-step:

$$\mathcal{Q}(\Theta | \theta) = \sum_{(p,r) \in \mathcal{D}} Q_{pr}(\Theta | \theta)$$

Using the results from your E-step, we can group the terms by the parameters they depend on:

$$\begin{aligned} \mathcal{Q}(\Theta) = & \sum_{p=1}^P \sum_{r \in R(p)} \underbrace{\left[-\frac{1}{2} \log(\sigma_p^2) - \frac{v_y^{(pr)} + (m_y^{(pr)} - \mu_p)^2}{2\sigma_p^2} \right]}_{\text{Depends on paper parameters}} \\ & + \sum_{r=1}^R \sum_{p \in P(r)} \underbrace{\left[-\frac{1}{2} \log(\tau_r^2) - \frac{v_z^{(pr)} + (m_z^{(pr)} - \nu_r)^2}{2\tau_r^2} \right]}_{\text{Depends on reviewer parameters}} + C \end{aligned}$$

(Note: We drop the tildes on the parameters here for cleaner notation, but remember these are the “new” parameters we are solving for, while m_y, v_y, m_z, v_z are constants calculated using the “old” parameters).

2.1.1 Updating Paper Intrinsic Value (μ_p)

To find the optimal intrinsic value for paper p , we take the partial derivative of $\mathcal{Q}$ with respect to μ_p . We only need to look at the sum over the reviewers $R(p)$ who read this specific paper.

$$\begin{aligned} \frac{\partial \mathcal{Q}}{\partial \mu_p} &= \sum_{r \in R(p)} -\frac{1}{2\sigma_p^2} \cdot 2(m_y^{(pr)} - \mu_p)(-1) \\ \frac{\partial \mathcal{Q}}{\partial \mu_p} &= \frac{1}{\sigma_p^2} \sum_{r \in R(p)} (m_y^{(pr)} - \mu_p) = 0 \end{aligned}$$

Since $1/\sigma_p^2 \neq 0$, the summation must equal zero. We can separate the sum:

$$\begin{aligned} \sum_{r \in R(p)} m_y^{(pr)} - \sum_{r \in R(p)} \mu_p &= 0 \\ \sum_{r \in R(p)} m_y^{(pr)} &= N_p \mu_p \end{aligned}$$

Update for μ_p :

$$\mu_p^{(new)} = \frac{1}{N_p} \sum_{r \in R(p)} m_y^{(pr)}$$

Intuition: The new estimate for the paper's true value is simply the average of the posterior means (m_y) across all its reviews.

2.1.2 Updating Paper Variance (σ_p^2)

Next, we take the partial derivative with respect to the variance σ_p^2 :

$$\frac{\partial \mathcal{Q}}{\partial (\sigma_p^2)} = \sum_{r \in R(p)} \left[-\frac{1}{2\sigma_p^2} + \frac{v_y^{(pr)} + (m_y^{(pr)} - \mu_p^{(new)})^2}{2(\sigma_p^2)^2} \right] = 0$$

Multiply the entire equation by $2(\sigma_p^2)^2$ to clear the denominators:

$$\sum_{r \in R(p)} \left[-\sigma_p^2 + v_y^{(pr)} + (m_y^{(pr)} - \mu_p^{(new)})^2 \right] = 0$$

Bring the σ_p^2 terms to the other side:

$$N_p \sigma_p^2 = \sum_{r \in R(p)} \left[v_y^{(pr)} + (m_y^{(pr)} - \mu_p^{(new)})^2 \right]$$

Update for σ_p^2 :

$$(\sigma_p^2)^{(new)} = \frac{1}{N_p} \sum_{r \in R(p)} \left[v_y^{(pr)} + (m_y^{(pr)} - \mu_p^{(new)})^2 \right]$$

Intuition: The new variance includes both our uncertainty from the E-step (v_y) plus the squared deviation of the posterior means from our new overall mean.

The Calculus Derivation

In the previous step, we did the *algebra* to solve for σ_p^2 . Here is the step-by-step *calculus* showing how we derived that equation from the original objective function.

Step 1: Isolate the relevant terms from the $\mathcal{Q}$ -function

The total expected log-likelihood $\mathcal{Q}$ contains many terms, but when we take the partial derivative with respect to σ_p^2 , any term that does not contain σ_p^2 is treated as a constant and its derivative becomes zero. The only terms in $\mathcal{Q}$ that contain σ_p^2 are inside the sum over the reviewers for paper p :

$$f(\sigma_p^2) = \sum_{r \in R(p)} \left[-\frac{1}{2} \log(\sigma_p^2) - \frac{v_y^{(pr)} + (m_y^{(pr)} - \mu_p^{(new)})^2}{2\sigma_p^2} \right]$$

Step 2: Treat σ_p^2 as a single variable

To make the calculus easier to see, let's substitute a dummy variable, let $x = \sigma_p^2$. Let $A = v_y^{(pr)} + (m_y^{(pr)} - \mu_p^{(new)})^2$ (since this entire numerator is just a constant with respect to σ_p^2). Now our function looks like standard Calculus 101:

$$f(x) = \sum_{r \in R(p)} \left[-\frac{1}{2} \log(x) - \frac{A}{2} x^{-1} \right]$$

Step 3: Apply standard derivative rules

Now we take the derivative with respect to x . The sum operator stays on the outside.

- The derivative of $\log(x)$ is $\frac{1}{x}$.
- The derivative of x^{-1} (using the power rule) is $-1 \cdot x^{-2} = -\frac{1}{x^2}$.

Applying these to our terms:

- Derivative of $-\frac{1}{2} \log(x)$ becomes $-\frac{1}{2x}$
- Derivative of $-\frac{A}{2} x^{-1}$ becomes $-\frac{A}{2} \left(-\frac{1}{x^2}\right) = +\frac{A}{2x^2}$

Putting it together inside the summation:

$$f'(x) = \sum_{r \in R(p)} \left[-\frac{1}{2x} + \frac{A}{2x^2} \right]$$

Step 4: Substitute the original variables back in

Replace x back with σ_p^2 , and replace A back with our numerator:

$$\frac{\partial \mathcal{Q}}{\partial (\sigma_p^2)} = \sum_{r \in R(p)} \left[-\frac{1}{2\sigma_p^2} + \frac{v_y^{(pr)} + (m_y^{(pr)} - \mu_p^{(new)})^2}{2(\sigma_p^2)^2} \right]$$

Finally, because we are looking for the maximum of the function, we set this derivative equal to 0, which gives you the exact equation!

2.1.3 Updating Reviewer Bias and Variance (ν_r and τ_r^2)

Because the generative model is perfectly symmetric between papers (y) and reviewers (z), the mathematical derivation for the reviewer parameters is identical. We just swap the variables and sum over all papers $P(r)$ reviewed by reviewer r .

Update for Reviewer Bias (ν_r):

$$\nu_r^{(new)} = \frac{1}{N_r} \sum_{p \in P(r)} m_z^{(pr)}$$

Update for Reviewer Variance (τ_r^2):

$$(\tau_r^2)^{(new)} = \frac{1}{N_r} \sum_{p \in P(r)} \left[v_z^{(pr)} + \left(m_z^{(pr)} - \nu_r^{(new)} \right)^2 \right]$$

2.1.4 Summary of the M-Step

In our algorithm, after calculating all the posterior expectations in the E-step, we will run these four equations to update our parameters. We will then feed these new parameters **back** into the E-step, repeating the cycle until the values converge. Once converged, our final μ_p values represent the fully “de-biased” intrinsic scores of the papers!

3. Reviewer Weighting

If a reviewer has a very large estimated τ_r^2 , their scores should be weighted significantly less in final decision-making.

3.1 Reasoning:

In our model, τ_r^2 represents the reviewer’s inconsistency. Mathematically, the posterior mean m_y (the paper’s true value estimate) is a weighted average of the observed scores. The “weight” or trust we place in a reviewer’s score is inversely proportional to their variance. When τ_r^2 is large, the posterior variance v_y for that specific reviewer’s contribution increases, effectively telling the model: “This reviewer is unreliable.” As a result, the EM algorithm automatically minimizes the impact of that reviewer’s data ($x^{(pr)}$) on the final estimate of μ_p . In practical terms, such reviewers are effectively ignored or “down-weighted” because their input adds noise rather than signal.